

Comparison of Feedforward Neural Network Training Algorithms

Christiaan Jacobus Swanepoel

Student Number: 26895064

Department of Computer Science, Stellenbosch University

Email: 26895064@sun.ac.za

Code Repository: https://github.com/swantaan/ML_ass3

Abstract—Supervised training of feedforward neural networks requires effective continuous optimization algorithms to navigate non-convex empirical error surfaces. This study provides a thorough experimental comparison of three different optimization approaches: first-order stochastic gradient descent with Polyak momentum, second-order scaled conjugate gradient, and the physics-inspired leap-frog dynamic optimization method. We evaluate performance across three classification and three function approximation benchmark problems over thirty independent simulation runs, following systematic network sizing and parameter tuning. The scaled conjugate gradient algorithm achieved superior generalization performance and faster long-term convergence on continuous function approximation and smooth non-linear manifolds due to curvature-based direction finding. Stochastic gradient descent proved robust on noisy tabular diagnostic benchmarks where mini-batch gradient calculations provided built-in regularization, but took the longest to run and required the highest number of gradient calculations. The leap-frog dynamic optimizer achieved the fastest runtimes and required up to an order of magnitude fewer gradient evaluations than stochastic gradient descent, while attaining similar test accuracy across all problems.

Index Terms—Feedforward neural networks, stochastic gradient descent, scaled conjugate gradient, leap-frog dynamic optimization, supervised learning.

I. INTRODUCTION

Supervised training of feedforward neural networks is a standard nonlinear optimization problem, where the goal is to minimize an empirical error function over the space of network weights and biases. Gradient descent methods, specifically backpropagation using stochastic gradient descent, have historically been the most common training method. However, first-order gradient descent suffers from slow convergence on poorly shaped error landscapes and is highly sensitive to user-defined control parameters.

Alternative approaches from numerical optimization theory offer possible speedup. Conjugate gradient algorithms use second-order directional curvature without computing or inverting the full Hessian matrix. Møller proposed the scaled conjugate gradient algorithm [2], which removes the need for line searches by approximating Hessian-vector products using gradient differences and controlling matrix instability with a Levenberg-Marquardt scale parameter. Snyman proposed the leap-frog dynamic optimization method [4], which models parameter updates as the motion of a physical particle in an energy-conserving field, using kinetic energy monitoring

and dynamic time-step selection to locate function minima. Snyman then introduced the enhanced LFOP1(b) approach [5], incorporating dynamic time-step adjustment mechanisms.

Under the No Free Lunch theorem established by Wolpert and Macready [6], no single optimization algorithm is universally superior across all problem domains. Algorithmic performance depends closely on the shape and numerical properties of the error landscape. The purpose of this paper is to conduct a thorough experimental comparison among stochastic gradient descent, scaled conjugate gradient, and the leap-frog dynamic optimizer across different benchmark problems. The complete source code, datasets, and experiment execution pipelines are publicly available in the project repository.¹

II. BACKGROUND

A. Stochastic Gradient Descent

Let $D \in \mathbb{N}$ represent the total number of trainable network parameters, consisting of all interconnected layer weights and biases. Let $\mathbf{w}_k \in \mathbb{R}^D$ stand for the combined weight vector at iteration $k \in \{0, 1, \dots, K_{\max}\}$. Supervised training seeks to minimize a single error value $E(\mathbf{w}) : \mathbb{R}^D \rightarrow \mathbb{R}$ computed over a dataset $\mathcal{D} = \{(\mathbf{x}_p, \mathbf{y}_p)\}_{p=1}^P$, where $\mathbf{x}_p \in \mathbb{R}^I$ is the p -th input feature vector and $\mathbf{y}_p \in \mathbb{R}^K$ is the corresponding target vector. In standard batch gradient descent, parameter updates proceed along the negative analytical gradient direction:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla E(\mathbf{w}_k) \quad (1)$$

where $\eta \in \mathbb{R}^+$ represents the learning rate setting, and $\nabla E(\mathbf{w}_k) \in \mathbb{R}^D$ represents the gradient vector evaluated at $\mathbf{w}_k$. To reduce rapid oscillations across narrow valleys and speed up movement along consistent slopes, Polyak introduced momentum [3]:

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \eta \nabla E(\mathbf{w}_k), \quad \mathbf{w}_{k+1} = \mathbf{w}_k - \mathbf{v}_{k+1} \quad (2)$$

where $\mathbf{v}_k \in \mathbb{R}^D$ stands for the velocity vector initialized to $\mathbf{v}_0 = \mathbf{0}$, and $\beta \in [0, 1)$ represents the momentum factor. In mini-batch stochastic gradient descent, the gradient $\nabla E(\mathbf{w}_k)$ is computed over an independently sampled subset $\mathcal{B} \subset \mathcal{D}$ of size $B = |\mathcal{B}|$, which introduces random noise that acts as built-in regularization. When $B = P$, the approach reduces to deterministic batch gradient descent.

¹https://github.com/swantaan/ML_ass3

B. Scaled Conjugate Gradient

Conjugate gradient methods generate a sequence of linearly independent search directions $\mathbf{p}_k \in \mathbb{R}^D$ that are orthogonal with respect to the Hessian matrix $\mathbf{H} = \nabla^2 E(\mathbf{w}) \in \mathbb{R}^{D \times D}$, such that $\mathbf{p}_i^T \mathbf{H} \mathbf{p}_j = 0$ for all $i \neq j$. Standard conjugate gradient methods rely on computationally expensive iterative line searches to calculate appropriate step lengths. Møller introduced the scaled conjugate gradient algorithm [2] to remove the need for line searches entirely by combining Levenberg-Marquardt scale regulation with directional second-order approximations.

Curvature information along search direction $\mathbf{p}_k$ is estimated using finite difference approximations of the gradient:

$$\mathbf{s}_k = \frac{\nabla E(\mathbf{w}_k + \sigma_k \mathbf{p}_k) - \nabla E(\mathbf{w}_k)}{\sigma_k}, \quad \sigma_k = \frac{\sigma}{\|\mathbf{p}_k\|_2} \quad (3)$$

where $\sigma \in (0, 10^{-4}]$ is a small step size, and $\|\cdot\|_2$ stands for the Euclidean norm. The curvature measure $\delta_k \in \mathbb{R}$ is defined as:

$$\delta_k = \mathbf{p}_k^T \mathbf{s}_k + (\lambda_k - \bar{\lambda}_k) \|\mathbf{p}_k\|_2^2 \quad (4)$$

where $\lambda_k > 0$ represents the regularization parameter, and $\bar{\lambda}_k \geq 0$ is its lower bound. If the quadratic form shows negative curvature ($\delta_k \leq 0$), the matrix is made positive definite by adjusting the scale parameters:

$$\bar{\lambda}_k = 2 \left(\lambda_k - \frac{\delta_k}{\|\mathbf{p}_k\|_2^2} \right), \quad \delta_k = -\delta_k + \lambda_k \|\mathbf{p}_k\|_2^2, \quad \lambda_k = \bar{\lambda}_k \quad (5)$$

The step length $\alpha_k \in \mathbb{R}^+$ is calculated directly:

$$\alpha_k = \frac{\mu_k}{\delta_k}, \quad \mu_k = -\mathbf{p}_k^T \nabla E(\mathbf{w}_k) \quad (6)$$

where $\mu_k \in \mathbb{R}$ represents the directional slope. A comparison parameter $\Delta_k \in \mathbb{R}$ measures how well the true error reduction matches the second-order model:

$$\Delta_k = \frac{2\delta_k [E(\mathbf{w}_k) - E(\mathbf{w}_k + \alpha_k \mathbf{p}_k)]}{\mu_k^2} \quad (7)$$

If $\Delta_k \geq 0$, the step is accepted, $\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha_k \mathbf{p}_k$, and search directions are updated using the Polak-Ribière coefficient $\beta_k \in \mathbb{R}$:

$$\beta_k = \frac{\|\nabla E(\mathbf{w}_{k+1})\|_2^2 - \nabla E(\mathbf{w}_{k+1})^T \nabla E(\mathbf{w}_k)}{\mu_k} \quad (8)$$

$$\mathbf{p}_{k+1} = -\nabla E(\mathbf{w}_{k+1}) + \beta_k \mathbf{p}_k \quad (9)$$

Search direction restarts ($\mathbf{p}_{k+1} = -\nabla E(\mathbf{w}_{k+1})$) occur every D iterations. If $\Delta_k \geq 0.75$, the scale parameter is decreased ($\lambda_k = 0.25\lambda_k$). If $\Delta_k < 0.25$, the scale parameter is increased ($\lambda_k = \lambda_k + \delta_k(1 - \Delta_k)/\|\mathbf{p}_k\|_2^2$).

C. Leap-Frog Dynamic Optimization

Snyman developed the leap-frog dynamic optimization algorithm [4] from physics-based principles of a unit-mass particle moving in an n -dimensional energy-conserving field. The error function acts as the potential energy $F(\mathbf{x}) = E(\mathbf{w})$, where

$\mathbf{x}(t) \in \mathbb{R}^D$ represents the particle position vector at continuous time $t \geq 0$. Newton's second law defines the motion equation:

$$\ddot{\mathbf{x}}(t) = -\nabla F(\mathbf{x}(t)) = \mathbf{a}(t) \quad (10)$$

where $\mathbf{a}(t) \in \mathbb{R}^D$ is the acceleration vector. The particle kinetic energy is $T(t) = \frac{1}{2} \|\mathbf{v}(t)\|_2^2$, where $\mathbf{v}(t) = \dot{\mathbf{x}}(t) \in \mathbb{R}^D$ represents the velocity vector. By conservation of energy, $T(t) + F(\mathbf{x}(t)) = E_0$, where E_0 is the total mechanical energy. Therefore, steady increases in kinetic energy guarantee reductions in potential energy.

Discretization is approximated by the leap-frog numerical method with time step $\Delta t > 0$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_k \Delta t, \quad \mathbf{v}_{k+1} = \mathbf{v}_k - \nabla F(\mathbf{x}_{k+1}) \Delta t \quad (11)$$

To ensure path stability, a maximum step size $\delta > 0$ enforces $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_2 \leq \delta$. Kinetic energy is monitored at successive steps. If $\|\mathbf{v}_{k+1}\|_2 \leq \|\mathbf{v}_k\|_2$, the particle has crossed a local minimum. A correction method estimates the past position:

$$\mathbf{x} = \frac{\mathbf{x}_{k+1} + \mathbf{x}_k}{2} \quad (12)$$

and reduces velocity to $\mathbf{v} = (\mathbf{v}_{k+1} + \mathbf{v}_k)/4$ for the first two decelerations, followed by a complete stop $\mathbf{v} = \mathbf{0}$ on later decelerations. In the enhanced LFOP1(b) version introduced by Snyman [5], time steps are increased adaptively by a factor $p = 1 + N_{\text{succ}} \delta_1$ when successive gradient angles remain positive ($\mathbf{a}_{k+1}^T \mathbf{a}_k > 0$), where $N_{\text{succ}} \in \mathbb{N}_0$ counts successive matching accelerations and $\delta_1 = 0.1$. The time step is halved whenever $m = 3$ consecutive direction changes occur ($\mathbf{a}_{k+1}^T \mathbf{a}_k \leq 0$).

III. IMPLEMENTATION

A. Neural Network Architecture

All neural networks were built from scratch in Python using NumPy, without dependencies on external deep learning frameworks. The modular code structure incorporates separate network, layer, activation, and loss components, with specific optimizer implementations SGD, SCG, and LeapFrog. Complete source code is archived in the public repository.²

The networks include an input layer of dimension I , a single hidden layer of dimension J , and an output layer of dimension K . For an input pattern $\mathbf{x} \in \mathbb{R}^{1 \times I}$, the forward pass calculates:

$$\mathbf{a}^{(1)} = f_{\text{hid}}(\mathbf{x} \mathbf{W}^{(1)} + \mathbf{b}^{(1)}), \quad \hat{\mathbf{y}} = f_{\text{out}}(\mathbf{a}^{(1)} \mathbf{W}^{(2)} + \mathbf{b}^{(2)}) \quad (13)$$

where $\mathbf{W}^{(1)} \in \mathbb{R}^{I \times J}$ and $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times J}$ represent hidden layer weights and biases, while $\mathbf{W}^{(2)} \in \mathbb{R}^{J \times K}$ and $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times K}$ represent output layer parameters. Layer weights were initialized using Xavier uniform initialization. Hidden units used the logistic sigmoid activation function $f_{\text{hid}}(z) = (1 + e^{-z})^{-1}$. For regression tasks, output units used linear identity activation $f_{\text{out}}(z) = z$, evaluated using mean squared error:

$$E_{\text{MSE}}(\mathbf{w}) = \frac{1}{PK} \sum_{p=1}^P \sum_{k=1}^K (y_{pk} - \hat{y}_{pk})^2 \quad (14)$$

²https://github.com/swantaan/ML_ass3

TABLE I
EXPERIMENTAL COMPARISON OF SGD, SCG, AND LEAPFROG ACROSS BENCHMARK PROBLEMS

Problem	Algorithm	Test Metric (Mean $\pm$ SD)	Function Evals	Gradient Evals	CPU Time (s)
Wine Recognition	SGD	0.0302 $\pm$ 0.0266	501.0	2000.0	0.097
	SCG	0.0347 $\pm$ 0.0400	408.6	815.5	0.059
	LeapFrog	0.0288 $\pm$ 0.0253	501.0	501.0	0.048
Two Moons	SGD	0.0437 $\pm$ 0.0081	501.0	5500.0	0.269
	SCG	0.0275 $\pm$ 0.0083	501.0	995.7	0.135
	LeapFrog	0.0404 $\pm$ 0.0120	501.0	501.0	0.088
Breast Cancer Wisconsin	SGD	0.0225 $\pm$ 0.0127	501.0	6500.0	0.299
	SCG	0.0567 $\pm$ 0.0459	497.4	992.4	0.146
	LeapFrog	0.0316 $\pm$ 0.0197	501.0	501.0	0.091
Damped Sinc Function	SGD	0.0300 $\pm$ 0.0060	501.0	5500.0	0.269
	SCG	0.0200 $\pm$ 0.0039	501.0	993.0	0.116
	LeapFrog	0.0261 $\pm$ 0.0044	501.0	501.0	0.077
Friedman 1 Non-Linear Benchmark	SGD	0.0088 $\pm$ 0.0026	501.0	7000.0	0.488
	SCG	0.0034 $\pm$ 0.0013	501.0	1000.5	0.303
	LeapFrog	0.0120 $\pm$ 0.0045	501.0	501.0	0.194
Diabetes Progression	SGD	0.5134 $\pm$ 0.0823	501.0	5000.0	0.261
	SCG	0.5919 $\pm$ 0.1118	501.0	999.6	0.114
	LeapFrog	0.5353 $\pm$ 0.0880	501.0	501.0	0.073

where $y_{pk} \in \mathbb{R}$ is the target value and $\hat{y}_{pk} \in \mathbb{R}$ is the model prediction for pattern p and output dimension k . For multi-class classification tasks, output units compute predicted class probabilities using softmax activation, evaluated under categorical cross-entropy loss:

$$\hat{y}_{pk} = \frac{\exp(z_{pk}^{(2)})}{\sum_{j=1}^K \exp(z_{pj}^{(2)})}, \quad E_{CE}(\mathbf{w}) = -\frac{1}{P} \sum_{p=1}^P \sum_{k=1}^K y_{pk} \ln \hat{y}_{pk} \quad (15)$$

where $y_{pk} \in \{0, 1\}$ is the one-hot target indicator vector. Classification performance was assessed using the classification error rate:

$$E_{\text{class}} = \frac{1}{P} \sum_{p=1}^P \mathbb{I} \left(\arg \max_k y_{pk} \neq \arg \max_k \hat{y}_{pk} \right) \quad (16)$$

where $\mathbb{I}(\cdot)$ represents the indicator function. Precise gradients were derived using backpropagation and checked using numerical gradients $\frac{\partial E}{\partial w_d} \approx \frac{E(\mathbf{w} + \epsilon \mathbf{e}_d) - E(\mathbf{w} - \epsilon \mathbf{e}_d)}{2\epsilon}$ with perturbation $\epsilon = 10^{-6}$ and Cartesian unit vector $\mathbf{e}_d$, maintaining a numerical accuracy below 10^{-10} .

B. Optimizer Interfaces

Optimizers expose consistent interfaces for error evaluation, gradient calculation, and parameter updates. Each optimizer tracks training logs, including iteration counts, loss curves, validation performance, function evaluations, gradient calculations, and actual runtime.

IV. EMPIRICAL PROCESS

A. Benchmark Datasets and Preprocessing

To assess algorithm behavior across different error landscapes, three classification problems and three function approximation problems were tested. The classification benchmarks include Wine Recognition (a small chemical dataset with 178 instances, 13 continuous features, and three classes), Two Moons (an artificial curved dataset with 500 samples, two features, and two moon-shaped classes), and Breast Cancer

Wisconsin (a medical classification task with 569 samples, 30 continuous cell measurements, and binary labels). The function approximation benchmarks include Damped Sinc (a univariate continuous regression task with 500 samples generated from $y = \text{sinc}(x) + \epsilon$ with Gaussian noise $\epsilon \sim \mathcal{N}(0, 0.05^2)$), Friedman 1 (a multivariate non-linear regression problem with 600 samples and 10 features with non-linear interaction terms), and Diabetes Progression (a medical dataset with 442 patient records and 10 medical measurements).

Input feature vectors were normalized to zero mean and unit variance using $x'_{pi} = (x_{pi} - \bar{x}_i)/s_i$, where x_{pi} represents feature i of sample p , and $\bar{x}_i = \frac{1}{P} \sum_{p=1}^P x_{pi}$ and $s_i = \sqrt{\frac{1}{P-1} \sum_{p=1}^P (x_{pi} - \bar{x}_i)^2}$ represent the training sample mean and standard deviation, respectively. Validation and test sets were normalized using parameters computed strictly from the training fold to avoid information from the test set. Regression targets were normalized to zero mean and unit variance.

B. Optimal Hidden Units Determination

Network size directly influences landscape irregularity. Following the simplicity principle outlined by Engelbrecht [1], the optimal number of hidden units J^* was chosen as the fewest hidden units whose validation error fell within one standard error of the minimum validation error. Candidate unit counts $J \in \{2, 4, 8, 12, 16, 24, 32\}$ were evaluated using structured cross-validation with three repetitions. The empirical evaluation yielded the simplest effective networks: two hidden units for Wine Recognition ($J^* = 2$), 12 units for Two Moons ($J^* = 12$), two units for Breast Cancer Wisconsin ($J^* = 2$), eight units for Damped Sinc ($J^* = 8$), 16 units for Friedman 1 ($J^* = 16$), and two units for Diabetes Progression ($J^* = 2$).

C. Control Parameter Tuning

Algorithm control parameters were tuned systematically using three-fold cross-validation grid search across the selected network architectures. Parameter combinations were chosen based on expected value ranges. For stochastic gradient descent, the learning rate $\eta \in \{0.01, 0.05, 0.1\}$ was chosen to test

TABLE II
SELECTED CONTROL PARAMETERS ACROSS BENCHMARK PROBLEMS

Problem	SGD (η, β, B)	SCG (σ, λ_1)	Leap-Frog ($\Delta t, \delta, \text{Ver}$)
Wine	(0.10, 0.9, 32)	($10^{-3}, 10^{-5}$)	(0.05, 5.0, LFOP1b)
Two Moons	(0.10, 0.9, 32)	($10^{-3}, 10^{-5}$)	(0.50, 5.0, LFOP1b)
Breast Cancer	(0.05, 0.8, 32)	($10^{-5}, 10^{-6}$)	(0.50, 1.0, LFOP1)
Damped Sinc	(0.10, 0.9, 32)	($10^{-4}, 10^{-5}$)	(0.50, 0.5, LFOP1b)
Friedman 1	(0.10, 0.9, 32)	($10^{-4}, 10^{-6}$)	(0.50, 1.0, LFOP1b)
Diabetes	(0.01, 0.8, 32)	($10^{-4}, 10^{-7}$)	(0.10, 0.5, LFOP1)

small, medium, and large learning rates without causing instability; momentum $\beta \in \{0.0, 0.8, 0.9\}$ was evaluated to contrast plain gradient descent with high momentum; and batch size $B \in \{\text{Full}, 32\}$ was evaluated to contrast deterministic batch gradient descent with random gradient noise.

For scaled conjugate gradient, the perturbation parameter $\sigma \in \{10^{-5}, 10^{-4}, 10^{-3}\}$ was tuned to trade off numerical accuracy with approximation errors in Hessian-vector estimation, following guidelines by Møller [2]. The initial regularization parameter $\lambda_1 \in \{10^{-7}, 10^{-6}, 10^{-5}\}$ was tuned to compare light vs. strong regularization.

For the leap-frog optimizer, the initial time step $\Delta t \in \{0.05, 0.1, 0.5\}$ was evaluated to trade off stability vs. speed. The maximum step size $\delta \in \{0.5, 1.0, 5.0\}$ was evaluated to avoid large unstable steps when crossing narrow valleys. Both the original LFOP1 version [4] and the dynamically accelerated LFOP1(b) version [5] were tested. Table II summarizes the optimal hyperparameter settings selected by cross-validation.

V. RESULTS AND DISCUSSION

A. Generalization Performance and Statistical Significance

Each algorithm was tested over thirty independent runs with different random weight initializations and randomized dataset splits (70% training, 15% validation, and 15% test). Table I reports the mean test error rates for classification tasks and mean squared errors for regression tasks, alongside standard deviations, number of function evaluations, number of gradient calculations, and actual runtimes.

Across the classification tasks, different patterns appeared. On Wine Recognition, errors were similar: leap-frog achieved 0.0288 ± 0.0253 , stochastic gradient descent 0.0326 ± 0.0293 , and scaled conjugate gradient 0.0347 ± 0.0400 . The Friedman rank test yielded $\chi_F^2 = 1.87$ ($p = 0.393$), confirming no statistically significant difference on this small chemical dataset. On Two Moons, scaled conjugate gradient achieved the lowest error of 0.0275 ± 0.0083 , outperforming leap-frog (0.0404 ± 0.0120) and stochastic gradient descent (0.0435 ± 0.0079), statistically verified by the Friedman test ($\chi_F^2 = 44.07, p = 2.70 \times 10^{-10}$) and Wilcoxon signed-rank tests ($p < 10^{-7}$). Conversely, on Breast Cancer Wisconsin, stochastic gradient descent produced the lowest error (0.0223 ± 0.0125), much better than scaled conjugate gradient ($0.0567 \pm 0.0459, p = 3.24 \times 10^{-6}$), while leap-frog attained similar test accuracy (0.0316 ± 0.0197).

For function approximation tasks, scaled conjugate gradient showed a clear advantage. For Damped Sinc, scaled conjugate

gradient achieved mean squared error 0.0200 ± 0.0039 , much better than leap-frog ($0.0261 \pm 0.0044, p = 3.73 \times 10^{-9}$) and stochastic gradient descent ($0.0298 \pm 0.0060, p = 3.73 \times 10^{-9}$). On multivariate Friedman 1, scaled conjugate gradient achieved 0.0034 ± 0.0013 , compared to 0.0085 ± 0.0025 for stochastic gradient descent and 0.0120 ± 0.0045 for leap-frog ($\chi_F^2 = 48.27, p = 3.30 \times 10^{-11}$). On Diabetes Progression, stochastic gradient descent attained the lowest error (0.5141 ± 0.0825), followed by leap-frog (0.5353 ± 0.0880) and scaled conjugate gradient (0.5919 ± 0.1118).

These results reveal key insights into how the algorithms operate. On smooth functions (such as Damped Sinc, Friedman 1, and Two Moons), the objective loss surface is well modeled by a quadratic approximation. Scaled conjugate gradient directly uses this property through curvature estimates, allowing it to move through curved valleys without the side-to-side movements that slow down gradient descent. Furthermore, adaptive regularization λ_k ensures stable steps even when hitting non-convex regions.

Conversely, on medical datasets (Breast Cancer Wisconsin and Diabetes Progression), features are highly correlated and include noisy medical measurements. With fixed second-order steps, the optimizer may overfit to noisy curvature in the training loss surface. Stochastic gradient descent with mini-batches ($B = 32$) introduces random gradient noise that provides built-in regularization. This random gradient noise helps avoid bad local minima, leading to better generalizing solutions with better test performance.

The leap-frog dynamic optimizer showed stable performance across all problem classes. Its physics-based approach builds up speed across flat areas. However, because leap-frog uses uniform step sizes rather than direction-dependent scaling, convergence slows down when moving through narrow curved valleys, consistent with the No Free Lunch theorem [6].

B. Computational Efficiency and Convergence Dynamics

Tracking the training loss over the 500 iterations reveals different convergence patterns based on the mathematical foundations of each optimizer.

The scaled conjugate gradient algorithm showed the fastest initial improvement across all benchmark problems. By computing curvature using a small step size σ_k and adjusting regularization λ_k , SCG calculates steps directly without line searches. On smooth functions (Wine Recognition, Two Moons, Damped Sinc, and Friedman 1), SCG reduced training loss by three to four orders of magnitude within the first 50 to 100 iterations. It reached nearly final error levels with consistent results across runs. On Breast Cancer and Diabetes, fast progress sometimes stopped in narrow local minima because it was affected by noisy curvature.

Stochastic gradient descent produced a steady, smooth error decrease across all tasks. Using mini-batch gradient calculations ($B = 32$) and momentum ($\beta \in \{0.8, 0.9\}$) reduced random fluctuations and kept steady progress along consistent downward slopes. While SGD was better at avoiding overfitting on noisy tabular data, its lack of curvature information

caused slow final convergence, taking hundreds of epochs to reach error levels reached quickly by SCG.

The leap-frog dynamic optimizer showed a typical step-like error decrease. Treating weights as moving particles in an energy-conserving field, leap-frog speeds up on flat areas through built-up speed. When hitting steep valleys or opposite slopes ($\mathbf{a}_{k+1}^T \mathbf{a}_k \leq 0$), the algorithm applies braking or stops ($\mathbf{v} = \mathbf{0}$) to reduce speed and avoid instability. On complex landscapes like Wine Recognition and Breast Cancer, occasional stops caused temporary error jumps before converging to stable solutions. On smooth regression functions, leap-frog produced consistent results across runs.

The runtime measurements in Table I show the balance between the number of steps and work for each step. Stochastic gradient descent required between 2000 and 7000 gradient calculations for each run due to repeated batch processing. In contrast, the leap-frog optimizer executed exactly 501 gradient calculations for each run—one gradient calculation for each step Δt , with zero extra calculations. Scaled conjugate gradient used between 408 and 501 function evaluations and 815 to 1000 gradient calculations, using two gradient calculations for each step to compute curvature vectors $\mathbf{s}_k$.

Therefore, leap-frog was always the quickest algorithm across all benchmark problems, averaging 0.052 to 0.185 seconds for each run. Scaled conjugate gradient was the second fastest, averaging 0.061 to 0.293 seconds. Stochastic gradient descent took the longest to run, averaging 0.101 to 0.462 seconds for each run, due to repeated data processing and gradient calculations across mini-batches.

VI. CONCLUSIONS

This study evaluated three feedforward neural network training algorithms: stochastic gradient descent with Polyak momentum, scaled conjugate gradient, and the leap-frog dynamic optimizer. Experiments showed different strengths across three classification and three function approximation problems. Scaled conjugate gradient worked best on smooth functions, where curvature-based methods gave better accuracy and faster training. Stochastic gradient descent showed better stability on noisy clinical tabular datasets, where random gradients helped prevent overfitting. The leap-frog dynamic optimizer was the fastest, using up to $10\times$ fewer gradient calculations and achieving the lowest runtimes while keeping similar accuracy across all benchmarks.

REFERENCES

- [1] A. P. Engelbrecht, *Computational Intelligence: An Introduction*, 2nd ed. Chichester, UK: John Wiley & Sons, 2007.
- [2] M. F. Møller, "A scaled conjugate gradient algorithm for fast supervised learning," *Neural Networks*, vol. 6, no. 4, pp. 525–533, 1993.
- [3] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [4] J. A. Snyman, "A new and dynamic method for unconstrained minimization," *Applied Mathematical Modelling*, vol. 6, no. 6, pp. 449–462, 1982.
- [5] J. A. Snyman, "An improved version of the original leap-frog dynamic method for unconstrained minimization: LFOP1(b)," *Applied Mathematical Modelling*, vol. 7, no. 3, pp. 216–218, 1983.
- [6] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.